

ACKNOWLEDGEMENT

As we have started this Project work with the sense of learning something on the topic “**Smart Poultry Farming with IoT and Machine Learning**”.

The euphoria and satisfaction that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible whose constant guidance and encouragement crowned our efforts with success.

First of all, we thank **Visvesvaraya Technological University, Belagavi** for providing this opportunity to implement this Project work.

We consider ourselves proud to be a part of "**P.D.I.T**" family, the institution which stood by our way in all endeavors.

We are extremely grateful to **Sri. Kalgudi Manjunath**, Chairman, PDIT, for providing all the required resources for the successful completion of the Project work.

We also take this opportunity to express our gratitude to **Dr. Rohitha U.M**, Principal of PDIT, for providing us environment to do the Project work.

We sincerely thank **Prof. Parvati Kadli HOD ,Dept. of CSE, PDIT**, without whose guidance and moral support this Project work would not have been successful.

We would like to express our heartfelt gratitude towards our inspiring project work **coordinator Dr. Manjula S D** whose firm belief in our capabilities and moral support to bring our potential to the forefront has played a major role in accomplishment of this Project work.

We thank **Project Guide Prof. Roopananda M K**, for guiding to us and We heartfully be thankful to our guide.

We would like to mention a thanks to all our **Teaching and Non-Teaching staff** members of Computer Science department PDIT, for their valuable support and guidance.

Last but not the least, We would like to thank our Parents and Friends for their support and suggestions without which we couldn't have achieved this success.

PROJECT ASSOCIATES

PREETHA M S

MADHU

SAHANA D

SANDHYA V

ABSTRACT

Poultry farming requires a controlled environment to ensure the optimal growth and health of chickens. This project presents an IoT-based Chicken Boarding Temperature Control System using ESP32 to monitor and regulate temperature, humidity, lighting, and air quality inside the chicken coop. The system integrates multiple sensors, actuators, and machine learning for early disease detection. The temperature and humidity sensors continuously monitor the environment, activating a heater (bulb), fan (cooler), and water pump to maintain optimal conditions. A gas sensor detects harmful gases like ammonia and methane, triggering alerts via a buzzer and GSM module if unsafe levels are detected. An LDR sensor helps automate lighting based on day and night cycles. The servo/DC motor-controlled door manages ventilation and security. Users can control and monitor the system via Bluetooth using a mobile app, while GSM notifications provide remote alerts in case of critical conditions. Additionally, a machine learning model is incorporated to analyze sensor data and predict early signs of diseases, helping in timely intervention. This smart poultry farm system enhances chicken welfare, reduces manual labor, and improves productivity by ensuring a fully automated and optimized environment.

CONTENTS

CHAPTER NO	PAGE NO
1. INTRODUCTION	
1.1 Introduction	01
1.2 Problem Statement	03
1.3 Existing System	04
1.4 Proposed System	06
2. OBJECTIVES	08
3. LITERATURE SURVEY	09
4. SOFTWARE REQUIREMENT SPECIFICATION	
4.1 Software Requirements	12
4.2 Hardware Requirements	13
5. SYSTEM DESIGN & ARCHITECTURE	
5.1 SYSTEM DESIGN	16
5.2 CIRCUIT DIAGRAM	17
6. IMPLEMENTATION	21
7. RESULTS AND SNAPSHOTS	23
8. ADVANTAGES AND APPLICATIONS	25
9. CONCLUSION	27
10. FUTURE SCOPE	28
REFERENCES	29

LIST OF FIGURES

SL.NO	NAME	PAGE NO
1.1	Smart Farm	2
1.2	Traditional Poultry Farming	5
4.1	ESP32	13
4.2	DHT11 Sensor	13
4.3	MQ135 Sensor	13
4.4	LDR Sensor	13
4.5	Ultrasonic Sensor	14
4.6	Servo Motor	14
4.7	DC Motor	14
4.8	Relay	14
4.9	Water Pump	14
4.10	LCD Display	14
5.1	Circuit Diagram	17
7.1	Complete hardware setup	23
7.2	Auto LED light ON in dark conditions	23
7.3	Water dispensing display on LCD	24
7.4	Cleaning display on LCD	24
7.5	Displaying Ammonia gas detection	24
7.6	LCD indicating real time temperature	24

CHAPTER 1

INTRODUCTION

The Smart Poultry Farming System developed in this project is designed to monitor and automate essential poultry farm operations using ESP32-based control and AI-based disease identification. The system focuses on maintaining proper environmental conditions, automating routine tasks such as feeding, watering, and cleaning, ensuring bird safety, and identifying poultry diseases using an image-based prediction model. Each part of the system operates through clearly defined processes, where the ESP32 handles the physical environment and the CNN model handles disease analysis.

The environmental monitoring section continuously observes temperature and humidity levels using the DHT11 sensor. These readings allow the system to determine whether the poultry environment is within the required range. When the temperature rises beyond the fixed limit, a cooling fan is turned on automatically using a relay, and when the values return to normal, the fan is turned off. This ensures that the birds remain in a comfortable climate at all times. The system also measures ammonia gas concentration using an MQ-type gas sensor. Poultry waste can release harmful gases, and when the sensor detects a high level, the system triggers a cleaning cycle using a motor-driven mechanism. This maintains hygiene and reduces the risk of infections. Every change in these conditions, along with the actions taken, is displayed on the 16×2 LCD for easy monitoring by the operator.

Lighting conditions inside the poultry area are handled using an LDR sensor. When the surroundings become dark, the system automatically switches on a light using a relay to maintain visibility for the birds. This is important for their movement, feeding behavior, and routine activity. Once the lighting improves naturally, the relay is turned off. For safety monitoring, an ultrasonic sensor continuously measures distance to detect the presence of any person or moving object near the poultry area. When movement is detected, a servo motor changes its position, and a notification message is sent to the owner via Telegram. This function helps alert the farmer about any activity near the poultry unit, especially during night hours or when the owner is away.

Routine poultry operations such as feeding and supplying water are also automated. The food supply mechanism uses a DC motor that turns forward and reverse for a short duration to dispense the required amount of food. The water supply is controlled through a relay-operated water pump that runs for a fixed time to fill the drinking tray. These operations occur automatically at regular intervals, making the system suitable for farms where manual feeding and watering may not always be consistent. The cleaning mechanism works using another motor that moves in both directions to simulate a cleaning cycle. This helps in maintaining hygiene

without constant manual effort.

The disease detection system allows poultry owners to check the health condition of a chicken by uploading an image. This part of the project uses a Convolutional Neural Network (CNN) model trained with a dataset of poultry images representing various conditions. The model is created and trained separately, and after training, it is used inside a Flask-based web application. When a user uploads an image through the website, the model analyzes the image and predicts whether the chicken is healthy or affected by a specific disease. The processed image outputs such as grayscale, sharpened, blurred, and edge-detected versions help in understanding different visual features. The prediction page also provides a description of the disease along with common symptoms and precautions.

Once a prediction is generated on the webpage, the result is sent through serial communication to the ESP32. Based on the received message, the controller displays the disease status on the LCD and sends a Telegram alert to the farmer. This makes it possible for the poultry owner to receive immediate information about the health condition of the bird without requiring direct inspection. All major events—temperature alerts, gas detection, cleaning completion, food supply status, water supply operation, and disease result notifications—are delivered to the farmer using the Telegram Bot API. This keeps the owner updated at every moment even if the person is not physically present near the poultry area.

The Smart Poultry Farming System therefore provides continuous monitoring, automatic environmental control, routine operation automation, and AI-based disease analysis. Every component works according to the required conditions and thresholds, ensuring that the poultry unit remains safe, clean, and properly managed. This approach reduces manual workload, increases reliability, and helps ensure that poultry health and farm operations are handled in a timely and efficient manner. The system is specifically designed to suit real-time poultry farm environments where quick detection, immediate response, and regular automation are essential for maintaining the well-being of the birds.



Fig 1.1: Smart Farm

1.1 PROBLEM STATEMENT

Poultry farms require continuous supervision to maintain appropriate temperature, humidity, lighting, hygiene, and food–water availability. In many farms, these responsibilities are handled manually, which often leads to inconsistent monitoring, delayed responses, and difficulty in maintaining stable environmental conditions. Temperature fluctuations, poor ventilation, ammonia gas buildup, and irregular feeding patterns can directly affect the health and growth rate of birds. Likewise, the absence of timely cleaning increases the risk of infections and reduces overall farm hygiene. These challenges are difficult to manage continuously without a reliable automated system.

Another major problem faced in poultry farms is the early identification of diseases. Poultry birds commonly develop infections that show visual symptoms, but identifying them in the early stage requires regular inspection and expertise. Manual inspection is not only time-consuming but also prone to human error. In many cases, diseases remain unnoticed until they spread widely within the flock, resulting in economic loss and health complications. Farmers do not always have access to immediate diagnostic support or tools that can help them examine the condition of a bird quickly and accurately.

Farmers also lack a notification system that can alert them instantly during emergencies such as high temperature, harmful gas levels, unexpected movement near the poultry area, or completion of feeding and cleaning cycles. Without real-time alerts, the response time to critical events becomes slow, especially when the farmer is away from the farm. The absence of remote monitoring increases the risk of overlooked issues, leading to unwanted situations that could have been prevented.

Therefore, there is a need for a system that can independently monitor environmental parameters, automate essential poultry operations, support disease identification through an image-based analysis method, and provide instant notifications for every important event. Solving these issues requires a reliable and continuous monitoring approach that can assist farmers in maintaining a stable, safe, and healthy poultry environment without depending entirely on manual supervision.

1.2 EXISTING SYSTEM

In the existing poultry farm setup, most activities are performed manually without the support of intelligent monitoring or automated mechanisms. Farmers rely on personal supervision to check temperature, humidity, lighting conditions, and hygiene levels inside the poultry shed. This method depends heavily on experience and continuous physical presence, making it difficult to maintain uniform conditions throughout the day. Temperature fluctuations often go unnoticed until they become severe, and humidity levels are rarely measured accurately. As a result, the environment inside the poultry area frequently shifts outside the ideal range, affecting the health, comfort, and growth of the birds.

Cleaning operations in traditional poultry systems also depend on scheduled manual activities. Waste accumulates quickly, producing harmful gases such as ammonia, but farmers usually become aware of this only when the smell becomes strong or visible discomfort appears in the birds. Without gas sensors or automatic cleaning mechanisms, harmful gases continue to build up, increasing the risk of infections, respiratory problems, and decreased productivity. Additionally, the cleaning process itself is time-consuming, labor-intensive, and often delayed due to manpower limitations or busy routines.

Lighting control is another area where traditional systems face challenges. Poultry require proper lighting for movement, feeding, and natural behavior. In existing setups, lights are manually switched on or off based on the farmer's judgment, which may not always align with the birds' requirements. If farmers forget to turn on the lights during dark conditions, the birds face difficulty in feeding. Similarly, unnecessary lighting during daytime results in energy wastage, as there is no automated light intensity detection.

Food and water management in conventional poultry farms operate without automation. Feeding is done by spreading food manually in intervals, which often leads to unequal distribution and inconsistent feeding times. Birds sometimes receive less food, while some areas receive excess, resulting in wastage. Water supply is maintained by filling containers manually, and farmers must frequently check the availability of water. If the farmer is late in refilling, the birds may remain without drinking water for long durations, affecting their routine and health.

Security or intrusion detection is very limited in the traditional system. Farmers usually inspect the poultry area only during specific times of the day. Any human or animal intrusion during odd hours goes unnoticed, which may lead to harm, theft, or disturbance to the birds. There is no automated mechanism to detect movement, alert the owner, or take preventive actions.

Disease identification in the existing system is entirely dependent on the farmer's observation. Birds often develop symptoms gradually, and early signs may not be visible to an untrained eye. Manual inspection requires

Smart Poultry Farming with IoT and Machine Learning

constant attention, and even experienced farmers struggle to detect diseases in the initial stage. Since no image-based or automated detection method exists, diseases are discovered only after they spread widely, making treatment more difficult and increasing losses.

Communication and alerting systems are also absent in traditional poultry practices. If a temperature rise occurs, food gets depleted, water is empty, or ammonia levels increase, the farmer receives no immediate alert. Since there is no remote monitoring or notification feature, critical conditions remain unnoticed until physical inspection, which delays necessary action.

Overall, the existing poultry system relies completely on manual monitoring and decision-making. This leads to irregular maintenance, delayed responses, increased labor requirements, higher chances of errors, and limited ability to manage emergencies. The absence of automatic control, sensor-based monitoring, disease detection tools, and real-time alerts makes the existing system less reliable, slow, and inefficient for modern poultry farm requirements.

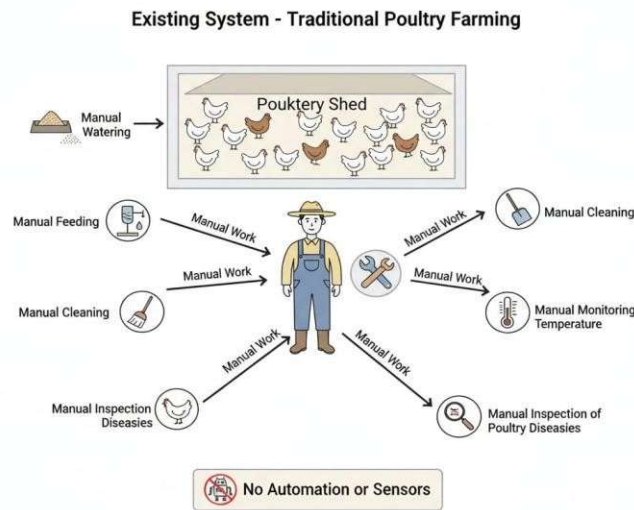


Fig 1.2: Traditional Poultry Farming

1.3 PROPOSED SYSTEM

The proposed Smart Poultry Farming System introduces an automated, sensor-based, and AI-assisted approach to overcome the limitations of conventional manual poultry management. The system focuses on continuous environmental monitoring, automatic control of farm operations, poultry disease prediction using a trained model, and real-time alerts to the farmer. Each function is designed to operate independently and efficiently to maintain a stable and healthy poultry environment while reducing manual effort and ensuring timely responses. In the proposed system, the ESP32 microcontroller is used as the main controller for managing all physical operations inside the poultry area. The temperature and humidity of the environment are continuously measured using the DHT11 sensor. These readings are displayed on an LCD and compared with predefined threshold values stored in the controller. When the temperature exceeds the safe operating range, the ESP32 automatically activates a cooling fan through a relay. Once the temperature returns to normal, the system switches off the fan. This keeps the environment regulated at all times without requiring human supervision.

To address hygiene and air quality, the proposed system includes an ammonia gas detection mechanism using an MQ-series gas sensor. When gas levels increase beyond a safe limit, the ESP32 activates a cleaning cycle through a motor-based cleaning mechanism that runs forward and reverse to remove accumulated waste. This prevents the buildup of harmful gases and ensures a cleaner and safer living space for the birds. The cleaning status and gas readings are shown on the LCD, and a notification is also sent to the farmer through Telegram to indicate that ammonia was detected and cleaning has been initiated.

Lighting management in the proposed system is automated through an LDR sensor. When the surrounding light level drops below a certain threshold, the system switches on the light using a relay to maintain proper visibility for the birds. Once the lighting conditions improve, the relay turns off the light automatically. This ensures that lighting conditions remain consistent with the needs of the birds without wasting energy.

The proposed system also automates routine tasks such as food and water supply. A DC motor-based feeding mechanism dispenses food at scheduled intervals by rotating forward and reverse for a specific duration. Water supply is handled through a relay-controlled pump that fills the water container automatically. Once the food and water supply cycles are completed, Telegram messages are sent to inform the farmer. This automation ensures timely feeding and consistent hydration, reducing dependency on manual labor and improving poultry management efficiency.

Security monitoring is enhanced through the use of an ultrasonic sensor that detects movement near the poultry area. When movement is detected within a certain range, the system activates a servo motor that opens or controls a designated mechanism, and an alert is sent immediately to the farmer via Telegram. This feature helps monitor any intrusion or unexpected activity inside the poultry area and provides timely notifications to improve safety, especially during nighttime or when the farmer is away.

A key feature of the proposed system is disease prediction using an AI-based Convolutional Neural Network (CNN) model. A Flask web application is used to allow the farmer to upload an image of a poultry bird. The trained model analyses the image and predicts whether the bird is healthy or affected by a specific disease. The webpage displays multiple processed versions of the uploaded image, such as grayscale, blurred, sharpened, and edge-detected outputs, along with the disease name, a short description, and symptoms. After generating the result, the Flask application sends a corresponding message to the ESP32 through serial communication. Based on this message, the controller displays the disease status on the LCD and sends a Telegram notification to the farmer, ensuring immediate awareness of the bird's condition.

The proposed system provides real-time event updates through the Telegram Bot API. Notifications are generated for temperature rise, gas detection, cleaning completion, food supply operation, water supply activity, and disease detection. These alerts ensure that the farmer stays informed at all times, even when not physically present at the poultry farm. The LCD connected to the ESP32 displays live readings and operation status, giving local visibility into system activities.

By combining automated monitoring, controlled operations, and AI-based disease identification, the proposed system ensures consistent environmental regulation, improved hygiene, timely feeding and watering, enhanced safety, and quick detection of diseases. The system provides a reliable and efficient solution that reduces manual effort and offers a smarter approach to poultry management compared to the traditional manual system.

CHAPTER 2

OBJECTIVES

1. To develop a Smart Poultry Farming System that automatically monitors and controls essential environmental conditions such as temperature, humidity, gas levels, light intensity, and movement.
2. To use the ESP32 microcontroller as the central control unit for processing sensor data and performing automated actions such as activating the fan, switching lights ON/OFF, cleaning the area, and detecting intruders.
3. To automate routine farm operations such as food supply, water supply, and cleaning using motors and relay mechanisms, reducing manual labor and ensuring consistent feeding and maintenance schedules.
4. To display real-time sensor readings and system activities on a 16×2 I2C LCD so that the farmer can easily monitor the status of the poultry environment.
5. To implement an AI-based poultry disease detection system using a trained CNN model that predicts whether a bird is healthy or diseased from an uploaded image.
6. To generate additional processed image outputs (grayscale, sharpened, blurred, edge-detected) to help the farmer visually understand the factors influencing the AI model's prediction.
7. To send disease prediction results to the ESP32 via serial communication so that the hardware system can update the LCD and trigger Telegram alerts based on the AI output.
8. To provide real-time notifications through Telegram, informing the farmer instantly about high temperature, ammonia detection, person intrusion, food/water supply activity, cleaning operation, and disease results.
9. To improve farm management efficiency by reducing manual intervention, increasing operational accuracy, and ensuring timely responses to unfavorable poultry conditions.
10. To maintain a safe, healthy, and stable living environment for the poultry throughout the day by integrating automation, AI, and real-time communication.

CHAPTER 3

LITERATURE SURVEY

Research in automated poultry management highlights two major areas: environmental monitoring and automatic control, and early disease detection using AI. Several studies emphasize that maintaining proper temperature, humidity, and air quality significantly reduces stress and disease among birds. Sensors such as DHT11/DHT22 and MQ-series gas sensors are widely used in automated systems to continuously track these parameters. Automatic actuation—such as turning ON fans, lights, pumps, or cleaning mechanisms—is commonly achieved using microcontrollers and relays, which offers greater accuracy than manual intervention.

The literature also supports automating routine operations like feeding and watering using DC motors and relay-based pumps. Proper light management using LDR sensors has been found important for healthy feeding cycles, while ultrasonic sensors have been shown effective for low-cost intrusion detection.

On the communication side, previous works show clear advantages of using mobile-based alerting systems (SMS, GSM, IoT dashboards, messaging apps) to notify farmers instantly about critical events. Platforms like Telegram are increasingly used for real-time, reliable alerts.

In disease detection research, Convolutional Neural Networks (CNNs) have proven highly effective for classifying poultry diseases based on images. Image preprocessing techniques—such as grayscale conversion, sharpening, and edge detection—are widely used to improve model accuracy. Lightweight web applications (e.g., Flask) are commonly used to allow farmers to upload images and view predictions. Many studies also recommend integrating AI predictions with microcontroller-based systems for real-time action.

A key gap identified in literature is the lack of integrated systems combining environmental automation + AI-based disease detection + instant communication + hardware actuation under a single platform. This project addresses this gap by unifying all these components using the ESP32, Flask, CNN model, Telegram alerts, and automated cleaning/feeding mechanisms.

Literature Survey of Existing Research Works

Sl. No.	Title / Author / Year	Key Focus	Methodology / Tools Used	Findings / Outcomes
1	Monitoring and Controlling of Poultry Farm Using IoT – Baig et al., 2017	IoT-based poultry automation	DHT11, LDR, fan & light control	Stable temperature & lighting control
2	Optimizing Poultry Disease Classification – Luo et al., 2025	Deep learning disease detection	CNN (ResNet, VGG16), image dataset	96.4% accuracy for 5 diseases
3	IoT-Based Poultry Farm Monitoring – Lifwarda & Srinivas, 2018	Cloud monitoring	DHT22, MQ135, LDR, ThingSpeak	Real-time tracking with cloud graphs
4	Smart Poultry Farm Using IoT – Himel & Islam, 2023	Remote monitoring	ESP8266, Blynk cloud	Accurate temperature & humidity logs
5	Automatic Poultry Farm System – Choukidar & Patil, 2017	Basic automation	Arduino, DHT11, relays	Good climate control; no alerts
6	IoT Broiler-House Monitoring – Fahrurrozi et al., 2024	Abnormal condition prediction	ML (RF, SVM), DHT22, MQ137	96.7% accuracy for predicting abnormal events
7	Wearable AI Poultry Behavior Detection – Ahmed et al., 2021	Activity-based disease detection	Wearable sensors, CT-GAN, TabNet	97% accuracy detecting infected birds
8	Precision Livestock Farming Review – Corkery et al., 2013	Overview of sensor-controlled poultry systems	Environmental sensors, lighting, feed control	Shows efficiency improvements in automated poultry
9	Automatic Poultry Climate Control – Ammad-Uddin et al., 2014	WSN-based monitoring	Temp, RH, gas sensors with SMS alerts	Improved flock health via automated climate control
10	Smart Poultry & PLF Review – Astill et al., 2020	Full IoT + AI review	Big data, cloud analytics, sensors	Identifies need for integrated IoT + ML systems
11	Real-Time Poultry	IoT temperature &	ESP8266, BME280	Reliable real-time

Smart Poultry Farming with IoT and Machine Learning

	Monitoring – Kumar & Rao, 2019	humidity monitoring		updates via web dashboard
12	Gas-Level Monitoring in Poultry Houses – Santos et al., 2020	Ammonia detection	MQ135, automated ventilation	Significant reduction in respiratory issues
13	Automated Feeding Mechanism for Poultry – Patel et al., 2018	Feeding automation	DC motor, timing mechanism	Reduced manual labor and consistent feed output
14	Light Intensity Control in Poultry – Raj & Devi, 2021	LDR-based lighting	LDR + relay switching	Improved feeding behavior and reduced stress
15	AI-Based Poultry Disease Detection Through Images – Sharma et al., 2022	Image-based ML	CNN, OpenCV preprocessing	Achieved >90% accuracy in classification

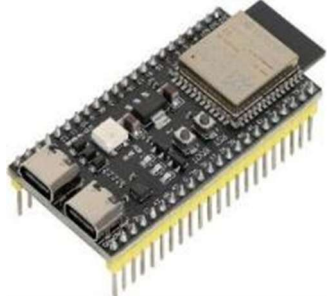
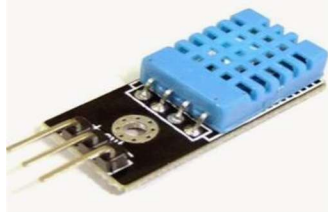
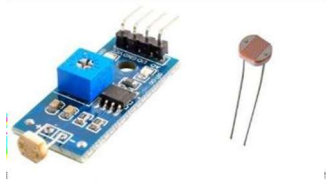
CHAPTER 4

SOFTWARE REQUIREMENT SPECIFICATION

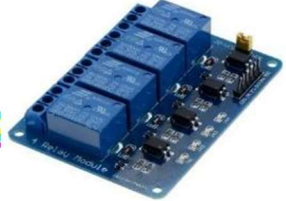
4.1 SOFTWARE REQUIREMENTS

Sl.No	Software Requirement	Description
4.1.1	Arduino IDE	Used to write, compile, and upload embedded C/C++ code to the ESP32. Supports libraries for Wi-Fi, Telegram, servo, LCD, and sensors. Includes Serial Monitor for debugging and real-time data observation.
4.1.2	Arduino Libraries	Arduino Libraries Includes WiFi.h, WiFiClientSecure.h, UniversalTelegramBot.h, LiquidCrystal_I2C.h, ESP32Servo.h, etc., enabling Wi-Fi communication, Telegram alerts, LCD display, servo control, and sensor interfacing.
4.1.3	Python Programming Language	Python Programming Language Used to develop the AI-based disease detection module. Supports dataset preprocessing, CNN training, prediction generation, and image processing operations. Communicates results to ESP32 via serial.
4.1.4	TensorFlow & Keras	TensorFlow & Keras Frameworks used for building and training the CNN model. Handle neural network design, optimization, and performance evaluation. Model is exported as H5 for integration with Flask.
4.1.5	OpenCV	OpenCV Performs image processing tasks such as grayscale, blur, sharpening, resizing, and edge detection. Prepares images for model prediction and generates processed outputs.
4.1.6	Flask Framework	Flask Framework Hosts the web application for image upload and disease prediction. Manages model loading, result display, and serial communication with the ESP32 using PySerial.
4.1.7	PySerial	PySerial Enables serial communication between Python and ESP32. Sends AI prediction results to the hardware for updating the LCD and sending Telegram alerts.
4.1.8	Telegram Application & Bot API	Telegram Application & Bot API Provides real-time notifications to the farmer. Bot API handles message delivery for temperature alerts, gas detection, feeding status, cleaning actions, and disease results.

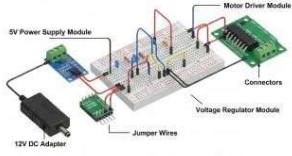
4.2 HARDWARE REQUIREMENT

Sl. No	Hardware Component	Description	Figure
4.2.1	ESP32 Microcontroller	ESP32 Microcontroller Central controller handling sensor data, Wi-Fi communication, motor control, LCD updates, and receiving AI results via serial. Supports multiple GPIOs, PWM, ADC, and dual-core processing.	 Fig 4.1: ESP32
4.2.2	DHT11 Sensor	DHT11 Sensor Measures temperature and humidity. Provides digital output to ESP32. Triggers cooling fan when temperature exceeds limit and sends updates to LCD & Telegram.	 Fig 4.2: DHT11 SENSOR
4.2.3	MQ Gas Sensor(MQ135)	MQ Gas Sensor (MQ135) Detects harmful gases, mainly ammonia. When threshold exceeded, triggers cleaning mechanism, updates LCD, and sends Telegram alert.	 Fig 4.3: MQ135 SENSNOR
4.2.4	LDR Sensor	LDR Sensor Measures light intensity. When it becomes dark, ESP32 switches ON the light automatically using a relay.	 Fig 4.4: LDR SENSOR

Smart Poultry Farming with IoT and Machine Learning

4.2.5	Ultrasonic Sensor	Ultrasonic Sensor Detects person/object intrusion based on distance measurement. Triggers servo movement and sends security alert to Telegram.	 Fig 4.5: ULTRASONIC SENSOR
4.2.6	Servo Motor	Servo Motor Performs angular movement when intrusion is detected. Controlled through PWM from ESP32. Resets after alert.	 Fig 4.6: SERVO MOTOR
4.2.7	DC Motors(Feeding & Cleaning)	DC Motors (Feeding & Cleaning) Feeding motor dispenses food at intervals; cleaning motor removes waste when ammonia is detected. Controlled via motor drivers.	 Fig 4.7: DC MOTOR
4.2.8	Relay Modules	Relay Modules Used to switch high-power devices (fan, light, pump) ON/OFF safely. Provides electrical isolation for ESP32.	 Fig 4.8: RELAY
4.2.9	Water Pump	Water Pump Pumps water at scheduled intervals through relay control. Ensures continuous drinking water availability.	 Fig 4.9: WATER PUMP
4.2.10	16x2 I2C LCD Display	16×2 I2C LCD Display Displays real-time values such as temperature, humidity, gas levels, motor status, and prediction results. Uses I2C to reduce wiring.	 Fig 4.10: LCD DISPLAY

Smart Poultry Farming with IoT and Machine Learning

4.2.11	Power Supply & Supporting Components	Power Supply & Supporting Components Includes 5V/12V power supply, motor drivers, resistors, jumper wires, connectors, and breadboard for stable operation and safe connections.	
---------------	--------------------------------------	--	---

CHAPTER 5

SYSTEM DESIGN & ARCHITECTURE

5.1 SYSTEM DESIGN

The system is organized to perform continuous environmental monitoring, deterministic control actions, scheduled automation, and image-based disease classification with clear, dependable information flow. The high-level architecture is composed of three logical layers: the Sensing Layer, the Control Layer, and the Application/Communication Layer.

The Sensing Layer contains all physical sensors that observe the poultry environment. The DHT11 sensor measures temperature and relative humidity. The MQ-series gas sensor provides an analog reading proportional to ammonia/air quality. The LDR provides an analog value corresponding to ambient light intensity. The ultrasonic sensor (HC-SR04 type) measures distance to detect presence or intrusion. All sensors are positioned inside or near the poultry enclosure at locations that represent typical bird conditions (DHT11 and MQ in the breathing/occupied zone, LDR at representative light location, ultrasonic at the entrance).

The Control Layer is implemented with an ESP32 development board that executes the automation logic. The ESP32 continuously samples the sensors, evaluates thresholds, runs scheduled routines, drives actuators and relays, updates the local display, and handles serial input from the AI module. Actuators include DC motors for feeding and cleaning (driven through a motor driver), a servo motor for a door/indicator movement, relay-switched devices for a fan, lamp and water pump, and the I2C 16×2 LCD for local status. The control logic enforces safety (default OFF outputs), debouncing/timing for motors, and immediate override actions when critical sensor thresholds are crossed.

The Application/Communication Layer contains two parts. Locally, a Flask web application loads the pre-trained Convolutional Neural Network model for poultry disease prediction; users upload an image and receive a prediction and processed image outputs (grayscale, sharpened, edged). The Flask app sends a compact serial message to the ESP32 to communicate the AI result (e.g., “A” = disease detected, “B” = healthy). Remotely, the ESP32 uses its onboard Wi-Fi and the Telegram Bot API to send real-time notifications to the farmer for events such as high temperature, ammonia detection, feeding/watering completion, intrusion detection and disease alerts. The Telegram alerts provide mobile visibility while the LCD provides on-site visibility.

Data flow is deterministic and event-driven. Sensor sampling and scheduled tasks run on the ESP32 main loop. When a sensor crosses a threshold, the ESP32 triggers the appropriate actuator, updates the LCD, and immediately sends a Telegram message. When the Flask app produces a disease prediction, it uses serial communication to inform the ESP32; the ESP32 treats that input like any other event and issues notifications and displays the result. No long-term database is used in the deployed system; all data are processed in real time and only the model file (poul.h5) and class label mapping are persisted for prediction.

The architecture emphasizes modularity and fault-tolerance. Each functional block (environment monitoring, feeding, cleaning, security, AI prediction, notification) is implemented as an independent module in firmware or software, with well-defined interfaces. Power distribution is segregated (low-voltage logic to ESP32 and sensors; separate supply to motors/relays) and common grounds are used. Safety considerations include using relay isolation for mains or high-current devices, motor drivers for DC motors, and ensuring the ESP32 GPIOs are never exposed to loads that exceed their ratings.

5.1.1 System Block Overview

ESP32 (central controller) ← receives → DHT11 (temp/humidity), MQ gas (analog), LDR (analog), Ultrasonic (trigger/echo)

ESP32 → controls → Relay1 (Fan), Relay2 (Light), Relay4 (Water Pump), Motor driver (Food motor IN1/IN2), Motor driver (Cleaning motor IN3/IN4), Servo (door)

ESP32 ↔ I2C LCD (16×2) for local display

Flask web app (PC/Server) → CNN model → predicts → sends serial result → ESP32

ESP32 → Telegram Bot (via Wi-Fi) → Farmer smartphone

5.1.2 Design principles applied

The system design follows these principles: single-responsibility modules (each sensor and actuator has a clearly defined function), event-driven alerts (immediate notification on threshold breaches), separation of concerns (embedded controller handles real-time control; Flask/CNN handles heavy image processing), safe electrical isolation for high-current devices (relays and motor drivers), and simplicity of wiring (I2C for LCD to reduce pins, ADCs for analog sensors). The design choices prioritize reliability, maintainability, and ease of deployment at small-to-medium farm scale.

5.1.3 Where and how components are placed in the poultry unit

The DHT11 and MQ sensor are mounted at bird height and away from direct influents (not directly above feeders or pumps) to represent general ambient conditions. The LDR is positioned to measure general enclosure illumination without being shadowed by structures. Ultrasonic sensor is installed at the entrance to detect incoming persons/objects. Motors and pump are located where they can mechanically reach feeders and cleaning apparatus; relays and the ESP32 are housed in a protected enclosure to avoid dust and moisture.

5.2 CIRCUIT DIAGRAM

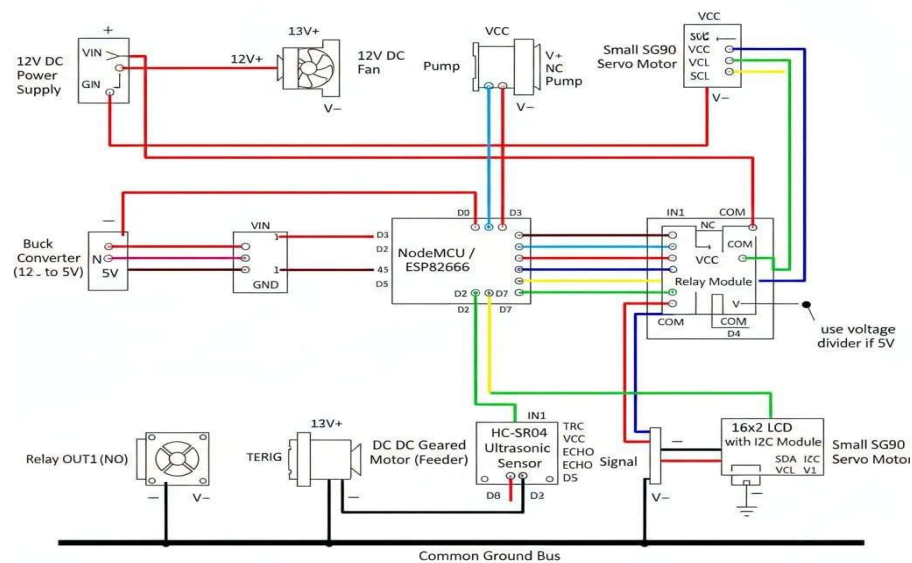


Fig 5.1: Circuit Diagram

The circuit uses a 12V DC supply to power the fan, water pump, and feeder motor through a 4-channel relay module, which allows the NodeMCU/ESP8266 to switch these high-power devices safely. A buck converter steps down 12V to 5V, which powers the NodeMCU, the ultrasonic sensor, the servo motor, and the 16×2 I2C LCD.

The NodeMCU sends control signals to the relays (IN1–IN4) and reads distance data from the HC-SR04 sensor (TRIG/ECHO). A servo motor is used for mechanical movement (such as door/flap control). All components share a common ground bus to ensure stable operation, and the ultrasonic ECHO pin uses a voltage divider for safe logic-level matching.

5.2.1 High-level wiring and power guidance

Power must be provided with appropriate supplies: the ESP32 and sensing logic operate at 3.3–5V (ESP32 typically powered via 5V USB or regulated 5V → 3.3V onboard regulator), logic-level relays and sensors often require 5V, while motors and water pump usually require a separate 12V (or motor-specific) supply. Use a common ground between all supplies (ESP32 ground, motor supply ground, relay ground) to ensure ADC readings and serial signals are stable. Use motor drivers (e.g., L298N or a MOSFET-based driver) between the ESP32 and DC motors; do not drive motors directly from GPIO pins. For mains AC loads (if fan or pump is AC), use a proper relay module rated for the AC current and ensure mains wiring complies with safety standards.

Place flyback diodes or use motor driver modules with built-in protection to protect the ESP32 from inductive spikes. Add decoupling capacitors near sensors and motor drivers to reduce transient noise that can affect ADC readings

5.2.2 Pin mapping

The following pin mapping is taken from the project code and recommended wiring. If you alter pins in the code, mirror those changes on the wiring.

ESP32 pin → Component / Signal

GPIO 23 → DHT11 data pin (digital input)

ADC 39 → MQ gas sensor analog output (gas reading)

ADC 36 → LDR (light sensor) analog output (note: ADC channel for GPIO36)

GPIO 32 → Ultrasonic Trigger (trigPin)

GPIO 33 → Ultrasonic Echo (echoPin)

GPIO 15 → Servo signal (servo control PWM)

GPIO 14 → Motor A IN1 (feeding motor forward)

GPIO 27 → Motor A IN2 (feeding motor reverse)

GPIO 26 → Motor B IN3 (cleaning motor forward)

GPIO 25 → Motor B IN4 (cleaning motor reverse)

GPIO 4 → Relay1 (Fan control)

GPIO 5 → Relay2 (Light control)

GPIO 18 → Relay3 (spare / optional)

GPIO 19 → Relay4 (Water pump control)

I2C SDA (typical GPIO 21) → LCD SDA (I2C)

I2C SCL (typical GPIO 22) → LCD SCL (I2C)

Serial2 (UART) → PC / Flask-hosted serial (for receiving AI results) — connect to USB-Serial on host as required

> Note: ADC pins (36, 39) are read as analog using the ESP32 ADC API; values may need scaling. Use the same ADC references your code expects.

5.2.3 Wiring detail for each component

DHT11: Connect Vcc to 5V (or 3.3V if using 3.3V DHT11), data pin to GPIO23, and ground to common ground. Add the recommended pull-up resistor (4.7k–10k) between data and Vcc if the sensor module does not include it.

MQ Gas Sensor: Connect Vcc to 5V, analog output (A0) to ADC39 on ESP32. Sensor modules often include a potentiometer for sensitivity and a digital threshold output; in this system the analog output is used. Place the sensor where gas accumulates but away from direct airflow from fans.

LDR: Use a voltage divider: LDR in series with a fixed resistor (e.g., 10k) between 3.3V and ground; take the divider midpoint to ADC36. Connect ground to common ground.

Ultrasonic Sensor (HC-SR04): Connect Vcc to 5V, TRIG pin to GPIO32, ECHO pin to GPIO33. Because ECHO can be 5V, use a voltage divider or level shifter on ECHO to bring pulses down to 3.3V safe for ESP32 input.

Servo Motor: Connect servo Vcc to 5V supply capable of the servo current (do not power servo from ESP32 3.3V pin), servo ground to common ground, and control signal to GPIO15. Add a decoupling capacitor across servo power if jitter or resets occur.

DC Motors (Feeding and Cleaning): Connect motors to a motor driver (L298N / appropriate driver) with separate motor power supply. Connect driver IN pins to the ESP32 GPIOs shown (IN1/IN2 → GPIO14/27; IN3/IN4 → GPIO26/25). Use enable pins on the driver for PWM speed control (if required). Ensure motor supply negative is tied to ESP32 ground.

Relay Modules: Each relay input connects to the respective ESP32 GPIO (GPIO4, GPIO5, GPIO18, GPIO19). Relay Vcc to 5V, relay ground to common ground. Use relay modules with opto-isolation if mains loads are used. Connect the relay common (COM) and normally open (NO) contacts to the device being switched (fan, lamp, pump). If switching AC mains, ensure wiring and insulation follow safety codes.

Water Pump: Connect to a relay contact controlling power to the pump. Use a pump rated for the supply voltage. Provide a separate power source sized for the pump and place the pump driver/relay inside an appropriate enclosure.

I2C LCD (16×2 with I2C backpack): Connect SDA to ESP32 SDA (GPIO21), SCL to SCL (GPIO22), Vcc to 5V or 3.3V per module requirements, ground to common ground. Address typically 0x27 (as used in code) — confirm with an I2C scanner.

Serial to Flask/PC: The Flask PC sends prediction messages over serial to the ESP32. Use USB-Serial or ESP32 Serial2 configured in code. If using a direct USB connection, ensure the host connection and COM port mapping are configured properly in the Flask app.

CHAPTER 6

IMPLEMENTATION

6.1 Hardware Implementation

The hardware implementation begins with the ESP32 microcontroller, which functions as the central control unit of the system. All sensors, actuators, and communication units are connected to the ESP32 through designated GPIO pins. The temperature and humidity readings are captured using the DHT11 sensor, which is positioned inside the poultry housing to measure environmental comfort. The MQ gas sensor is wired to the analog input of the ESP32 to detect ammonia concentration produced from poultry waste, while the LDR module is used for monitoring ambient light levels. An ultrasonic sensor is integrated for object and intrusion detection by measuring the distance of any moving object approaching the poultry enclosure.

To enable automated operations, multiple actuators are connected. The food dispensing mechanism is controlled using a DC motor connected through a motor driver module. This driver enables bidirectional rotation for proper feeding movement. The cleaning mechanism uses another DC motor connected through the same type of driver to push and pull waste as needed. Relays are incorporated to control high-power devices such as the cooling fan, water pump, and artificial lighting system. These relays act as switches that the ESP32 triggers based on sensor conditions. A servo motor is installed as part of the intrusion alert mechanism, offering controlled angular motion through a PWM pin on the ESP32.

The LCD display is connected via the I2C protocol, which simplifies wiring and reduces the number of pins required. It displays real-time information such as temperature, humidity, gas concentration, feeding status, and cleaning notifications. Power distribution is carefully planned using a regulated supply that provides stable 5V to the ESP32, sensors, relays, and LCD module. Motors and pumps are supplied separately to prevent power fluctuations. A common ground is maintained across all components to ensure stable operation and noise-free sensor readings. Once all wiring was completed, every module was individually tested to confirm proper functioning before assembling the entire system inside the poultry unit.

6.2 Software Implementation

The software implementation consists of the embedded firmware written for the ESP32 and the Python-based software for disease detection. The ESP32 program is developed using the Arduino IDE, where all sensor pins, motor driver signals, and relay outputs are initialized. The code begins by establishing a Wi-Fi connection using the farm network credentials. Once connected, the ESP32 interacts with the Telegram Bot API to send instant notifications to the farmer's mobile phone.

Inside the main program loop, the ESP32 continuously reads temperature, humidity, gas levels, light intensity, and ultrasonic distance values. Each reading is compared against predefined threshold values, and the system triggers the appropriate action. For example, when the temperature exceeds the limit, the fan relay is activated; when gas concentration rises, the cleaning motor is started; and when the LDR detects darkness, lighting is automatically turned on. The feeding and watering actions are controlled through predefined time-based cycles, ensuring consistent availability of food and water for poultry. All these operations are displayed on the LCD, and Telegram alerts are sent to keep the farmer updated in real time.

The software implementation also includes the AI-based disease detection module developed using Python, Flask, OpenCV, and a pre-trained Convolutional Neural Network (CNN). The Flask web application allows the

Smart Poultry Farming with IoT and Machine Learning

user to upload a poultry image. The image undergoes preprocessing steps such as grayscale conversion, edge detection, and noise removal. The CNN model then classifies the image as either “healthy” or “diseased.” Once the prediction is generated, the Flask application sends the result to the ESP32 via serial communication. The ESP32 receives this message, displays the result on the LCD, and forwards the information via Telegram. This seamless integration of machine learning with embedded systems completes the intelligent functionality of the project.

6.3 Database Implementation

The Smart Poultry Farming System is designed to operate in real time and does not store historical data; therefore, no dedicated external database is used. All sensor readings and automation decisions occur instantaneously within the ESP32 microcontroller’s memory. The values are displayed on the LCD and, in critical situations, sent directly to the farmer’s mobile phone through Telegram. Since the system’s purpose is immediate environment management rather than long-term analytics, database storage is not required.

Similarly, the AI-based disease detection module does not store images or health records. When a poultry image is uploaded through the Flask web interface, the image exists only in temporary memory for processing. The CNN model, trained using the TRAIN.py script, is loaded from a saved .h5 file. The prediction result is transmitted to the ESP32 without being stored in a database. This approach keeps the system lightweight, fast, and suitable for continuous operation without excessive storage overhead.

CHAPTER 7

RESULTS AND SNAPSHOTS

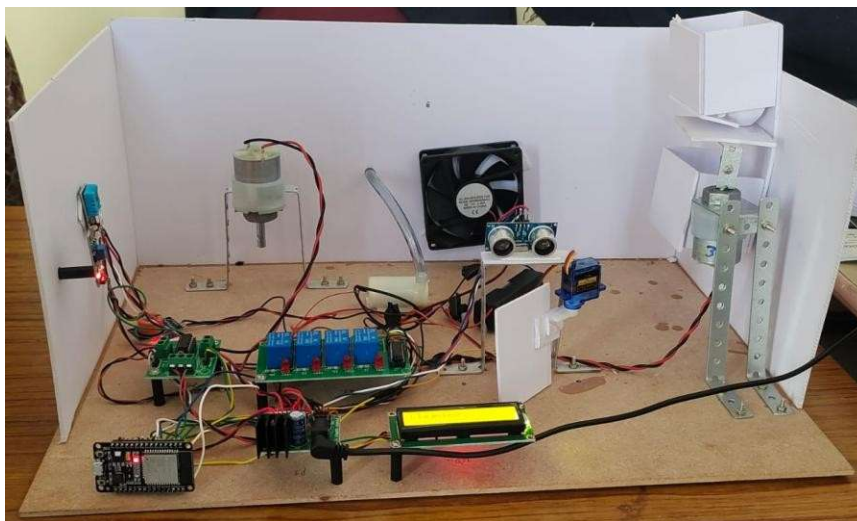


Fig 7.1: Complete hardware setup of the smart poultry farm prototype



Fig 7.2: Auto LED light ON in dark conditions



Fig.7.3 Water dispensing time displayed on LCD



Fig.7.4 Cleaning mode status shown on LCD



Fig. 7.5: LCD indicating real-time ammonia gas



Fig.7.6 LCD indicating real-time temperature values

CHAPTER 8

ADVANTAGES AND APPLICATIONS

8.1 ADVANTAGES

The Smart Poultry Farming System offers several important advantages that significantly improve the efficiency, accuracy, and safety of poultry farm management. One of the major benefits is the automation of environmental monitoring, which reduces dependency on manual inspection and ensures that the birds always remain in healthy conditions. By using sensors to measure temperature, humidity, gas concentration, and lighting levels, the system can make accurate real-time decisions that help maintain a stable environment, leading to healthier and more productive poultry.

The integration of automatic fan control, water management, food dispensing, lighting adjustment, and cleaning operations eliminates the need for frequent human intervention. This not only reduces labour costs but also minimizes human error, ensuring a consistent and reliable setup. The system also improves hygiene by automatically detecting waste and activating the cleaning mechanism, which prevents the spread of infections and reduces mortality rates in poultry farms.

Another major advantage is the inclusion of AI-based disease detection. By analysing bird images through a trained neural network model, the system can identify early signs of diseases that might otherwise go unnoticed. This allows farmers to take immediate corrective action, preventing the disease from spreading within the flock. The use of Telegram alerts further enhances the system's practicality by informing the farmer instantly about abnormal environmental conditions, cleaning activity, gas detection, and security breaches.

The solution is easy to scale and can be adapted for poultry farms of different sizes. It is cost-effective compared to traditional manual systems or fully commercial solutions, making it suitable even for small-scale farmers. Since the system relies on IoT components and open-source software, it can be easily updated, expanded, and customized according to specific requirements, providing long-term flexibility and future improvement opportunities.

8.2 APPLICATIONS

The Smart Poultry Farming System is designed for real-world use and can be applied in various agricultural settings. It is highly suitable for small-scale poultry farms that require inexpensive but reliable automation to improve productivity and reduce operational challenges. This system helps farmers maintain consistent feeding, watering, and hygiene schedules without needing to be physically present at all times, making it ideal for rural areas where manpower is limited.

Medium-scale and commercial poultry farms can also benefit from this system by using it to improve environmental control across larger poultry sheds. Continuous monitoring helps prevent losses caused by heat stress, insufficient ventilation, contamination, or delayed cleaning routines. The AI-based disease detection component is particularly beneficial for large farms where manual inspection of each bird is difficult and time-consuming.

Smart Poultry Farming with IoT and Machine Learning

Educational institutions, agricultural colleges, and research centres can use this system for training and experimentation. It provides a complete integration of IoT, embedded systems, automation, and machine learning, making it a valuable learning tool for students and researchers. The system can also be used by startups and agritech companies working on smart agriculture solutions to develop more advanced or commercial versions of automated poultry systems.

Overall, the Smart Poultry Farming System has wide applications across the poultry industry and contributes significantly to the modernization of agricultural practices, ensuring better productivity, reduced losses, and enhanced farm management through automation and intelligent monitoring.

CHAPTER 9

CONCLUSION

The Smart Poultry Farming System developed in this project provides an integrated, automated, and reliable solution for overcoming the challenges faced in traditional poultry management. By combining IoT-based monitoring with machine learning-based disease detection, the system successfully creates a controlled and intelligent environment that ensures better safety, hygiene, and productivity of poultry farms. The ESP32 microcontroller continuously collects real-time data from temperature, humidity, gas, LDR, PIR, and IR sensors, enabling the system to automatically regulate fans, lighting, water supply, feed dispensing, and cleaning operations without requiring human involvement. This automated process ensures stable environmental conditions, reduces labour, and minimizes the risk of human error.

The machine learning model implemented through a Flask application adds an intelligent layer to the system by detecting poultry diseases using image processing. By accurately identifying the health condition of birds based on their images, the model provides early warnings which help farmers take timely action and prevent the spread of infections. In addition, the Telegram alert system ensures that farmers receive instant notifications about feeding status, cleaning operations, abnormal temperature variations, gas detection, or any health issues detected by the AI model. This improves responsiveness and allows the farmer to monitor the poultry farm from anywhere.

Overall, the project has achieved all its goals by providing a complete automation system that reduces manual workload, improves accuracy in monitoring, enhances poultry welfare, and helps in maintaining better farm hygiene. The integration of hardware and software components has demonstrated the effectiveness of using IoT and artificial intelligence in agriculture. The system is practical, scalable, and suitable for real-time implementation in small, medium, and large poultry farms. This project clearly shows how modern technology can transform poultry management into a more efficient, sustainable, and intelligent process.

CHAPTER 10

FUTURE SCOPE

The Smart Poultry Farming System developed in this project provides a strong foundation for automated poultry management, yet there are several opportunities to enhance its functionality and scalability in future work. One significant improvement is the integration of cloud platforms such as Firebase or AWS IoT Core to store historical data of temperature, humidity, gas levels, feed cycles, and disease detection reports. This would enable long-term data analytics and predictive modeling to forecast environmental changes or identify disease patterns more accurately. Adding a mobile application with a user-friendly interface can further improve accessibility, allowing farmers to view real-time data, control devices manually when required, and receive visual insights directly on their smartphones.

The disease detection module can be expanded by training the model on a larger and more diverse dataset to identify multiple poultry diseases with higher accuracy. Advanced deep learning techniques such as transfer learning, YOLO-based real-time detection, or multi-class CNN classification can make the system more intelligent and capable of identifying early symptoms more precisely. The automation system can also be enhanced by adding features such as automated egg collection, weight monitoring of birds through load cells, and camera-based behaviour analysis using computer vision techniques.

Another important future scope is the adoption of renewable energy sources, such as solar panels, to power the entire system. This can reduce electricity consumption and make the solution more suitable for rural or remote poultry farms with limited access to power. Incorporating GSM or LoRaWAN communication will enable the system to function effectively even in areas without Wi-Fi connectivity. Finally, scaling the system to manage multiple poultry sheds simultaneously and enabling centralized monitoring can make the solution commercially viable for large-scale poultry industries.

REFERENCES

1. ESP32 Technical Reference Manual, Espressif Systems, 2023.
2. DHT11 Temperature and Humidity Sensor Datasheet, Aosong Electronics, 2022.
3. MQ Series Gas Sensors User Guide, Hanwei Electronics, 2021.
4. "Internet of Things (IoT) Applications in Agriculture," IEEE Access Journal, 2022.
5. "Deep Learning for Poultry Disease Detection Using CNN Models," International Journal of Computer Applications, 2023.
6. Python Flask Documentation – Pallets Projects, 2023.
7. OpenCV Library Documentation – OpenCV.org, 2023.
8. Universal Telegram Bot API Documentation, Telegram.org, 2024.
9. Ian Goodfellow, Yoshua Bengio, Aaron Courville, “Deep Learning,” MIT Press, 2016.
10. Arduino and ESP32 IoT Programming Guide, Arduino.cc, 2023.

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>
#include <ArduinoJson.h>

const char* ssid = "Poultry123";
const char* password = "123456789";

#define BOTtoken "8435852153:AAE4s0zgznUOhxeZObch6YHpnjvU5tCA5NQ" // your Bot Token (Get
from Botfather)
#define CHAT_ID "6021751414"

//X509List cert(TELEGRAM_CERTIFICATE_ROOT);
WiFiClientSecure client;
UniversalTelegramBot bot(BOTtoken, client);

char Start_buff[70];
int i,z;
char ch;
int str_len;
char textmessage[20];

#include <ESP32Servo.h>

Servo servo1;

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dht.h>

#define dht_dpin 23
```

```
int Relay1=4;
int Relay2=5;
int Relay3=18;
int Relay4=19;

#define trigPin 32
#define echoPin 33

int IN1=14;
int IN2=27;

int IN3=26;
int IN4=25;

long duration;
int distance;

dht DHT;

void setup()
{
  servo1.attach(15);
  servo1.write(90);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  pinMode(IN1,OUTPUT);
  pinMode(IN2,OUTPUT);
  pinMode(IN3,OUTPUT);
  pinMode(IN4,OUTPUT);

  pinMode(Relay1,OUTPUT);
  pinMode(Relay2,OUTPUT);
  pinMode(Relay3,OUTPUT);
  pinMode(Relay4,OUTPUT);

  digitalWrite(Relay1,LOW);
  digitalWrite(Relay2,LOW);
  digitalWrite(Relay3,LOW);
  digitalWrite(Relay4,LOW);

  digitalWrite(IN1,LOW);
  digitalWrite(IN2,LOW);
  digitalWrite(IN3,LOW);
```

```
digitalWrite(IN4,LOW);

Serial.begin(9600);
Serial2.begin(9600);
lcd.init();           // Initialize the LCD
lcd.backlight();
Serial.println("Smart Poultry Farming");
lcd.clear();
lcd.setCursor(0,0);
lcd.print("Smart Poultry");
lcd.setCursor(0,1);
lcd.print(" Farming...");
delay(2000);
Serial.println("Starting TelegramBot...");

Serial.print("Connecting Wifi: ");
Serial.println(ssid);

WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);
client.setCACert(TELEGRAM_CERTIFICATE_ROOT);

while (WiFi.status() != WL_CONNECTED) {
  Serial.print(".");
  delay(500);
}

Serial.println("");
Serial.println("WiFi connected");
Serial.print("IP address: ");
Serial.println(WiFi.localIP());
bot.sendMessage(CHAT_ID, "Smart Poultry Monitoring..");
delay(1000);

}

void loop()
{
  Food_Supply();
  Water_Supply();
  Cleaning();
  for(int z=0;z<20;z++)
  {
    Humidity_Check();
    LDR_Check();
    Gas_Check();
    Object_Check();
```

```
}
delay(1000);

}
void Cleaning()
{
  Serial.println("Cleaning");
  lcd.clear();
  lcd.print("Cleaning");
  delay(1000);
  digitalWrite(IN3,HIGH);
  digitalWrite(IN4,LOW);
  delay(2000);
  digitalWrite(IN3,LOW);
  digitalWrite(IN4,LOW);
  delay(1000);
  digitalWrite(IN3,LOW);
  digitalWrite(IN4,HIGH);
  delay(2000);
  digitalWrite(IN3,LOW);
  digitalWrite(IN4,LOW);
  bot.sendMessage(CHAT_ID, "Cleaning Process Completed");
  delay(1000);
  // SEND_SMS("+919353617078","Cleaning Process Completed");
  // delay(2000);

}
void Food_Supply()
{
  Serial.println("Food Time");
  lcd.clear();
  lcd.print("Food Time");
  delay(1000);
  digitalWrite(IN1,HIGH);
  digitalWrite(IN2,LOW);
  delay(500);
  digitalWrite(IN1,LOW);
  digitalWrite(IN2,LOW);
  delay(2000);
  digitalWrite(IN1,LOW);
  digitalWrite(IN2,HIGH);
  delay(500);
  digitalWrite(IN1,LOW);
  digitalWrite(IN2,LOW);
  delay(1000);
  bot.sendMessage(CHAT_ID, "Food Supply Completed");
  delay(1000);
  // SEND_SMS("+919353617078","Food Supply Completed");
  // delay(2000);
```

```
}
void Water_Supply()
{
  Serial.println("Water Time");
  lcd.clear();
  lcd.print("Water Time");
  delay(1000);
  digitalWrite(Relay4,HIGH);
  delay(3000);
  digitalWrite(Relay4,LOW);
  bot.sendMessage(CHAT_ID, "Water Supply Completed");
  delay(1000);
  // SEND_SMS("+919353617078","Water Supply Completed");
  // delay(2000);
}
void Object_Check()
{
  digitalWrite(trigPin, LOW);
  delayMicroseconds(5);

  // Trigger the sensor by setting the trigPin high for 10 microseconds:
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Read the echoPin, pulseIn() returns the duration (length of the pulse) in microseconds:
  duration = pulseIn(echoPin, HIGH);
  // Calculate the distance:
  distance = duration * 0.034 / 2;

  // Print the distance on the Serial Monitor (Ctrl+Shift+M):
  Serial.print("Distance = ");
  Serial.print(distance);
  Serial.println(" cm");
  lcd.clear();
  lcd.print("Distance= ");
  lcd.print(distance);
  lcd.print("cm");

  delay(1000);
  if(distance<50)
  {
    Serial.println("$Person Detected..#");
    lcd.clear();
    lcd.print("Person Detected..");
    servo1.write(0);
    delay(2000);
    servo1.write(90);
    bot.sendMessage(CHAT_ID, "Person Detected");
```

Smart Poultry Farming with IoT and Machine Learning

```
    delay(1000);
    // SEND_SMS("+919353617078","Person Detected");
    // delay(2000);
    }

}

char Serial_read(void)
{
    char ch;
    while(Serial.available() == 0);
    ch = Serial.read();
    return ch;
}

void Humidity_Check(void)
{
    DHT.read11(dht_dpin);
    Serial.print("Humidity: ");
    Serial.print(DHT.humidity); // printing Humidity on LCD
    Serial.print(" %");
    Serial.print(" ");

    delay(500);

    Serial.print("Temperature:");
    Serial.print(DHT.temperature); // Printing temperature on LCD
    Serial.print(" C");
    Serial.println(" ");

    lcd.clear();
    lcd.print("RH:");
    lcd.print(DHT.humidity);
    lcd.setCursor(0, 1);
    lcd.print("TEMP:");
    lcd.print(DHT.temperature);
    delay(1000);
    if(DHT.temperature >= 33.0)
    {
        lcd.clear();
        lcd.print("More Temp");
        lcd.setCursor(0, 1);
        lcd.print("Fan On..");
        Serial.println("$More Temperature Fan On..#");
        digitalWrite(Relay1, HIGH);
        bot.sendMessage(CHAT_ID, "More Temperature Fan On.");
    }
    delay(1000);
}
```

Smart Poultry Farming with IoT and Machine Learning

```
// SEND_SMS("+919353617078","More Temperature Fan On.");  
// delay(2000);
```

```
}  
if(DHT.temperature<33.0)  
{  
  lcd.clear();  
  lcd.print("Normal Temp");  
  lcd.setCursor(0, 1);  
  lcd.print("Fan Off.");  
  Serial.println("Normal Temperature Fan Off");  
  digitalWrite(Relay1,LOW);  
  delay(1000);  
}  
}
```

```
void Gas_Check()  
{  
  int Gas_Value=analogRead(39);  
  // Gas_Value=1023-Gas_Value;  
  Serial.print("Gas:");  
  Serial.println(Gas_Value);  
  lcd.clear();  
  lcd.print("Gas:");  
  lcd.print(Gas_Value);  
  delay(1000);  
  if(Gas_Value>3000)  
  {  
    Serial.println("$Amonia Gas Detected#");  
    lcd.clear();  
    lcd.print("Amonia Gas");  
    lcd.setCursor(0,1);  
    lcd.print("Detected");  
    bot.sendMessage(CHAT_ID, "Amonia Gas Detected");  
    delay(1000);  
    Cleaning();  
  }  
  //SEND_SMS("+919353617078","Amonia Gas Detected");  
  // delay(2000);  
}
```

```
}  
void LDR_Check()  
{  
  int LDR_Value=analogRead(36);  
  LDR_Value=4095-LDR_Value;  
  Serial.print("LDR:");  
  Serial.println(LDR_Value);  
  lcd.clear();  
  lcd.print("LDR:");  
  lcd.print(LDR_Value);
```



```
delay(1000);
if(LDR_Value<100)
{
    Serial.println("$Dark Mode Light On#");
    lcd.clear();
    lcd.print("Dark Mode");
    lcd.setCursor(0,1);
    lcd.print("Light On");
    digitalWrite(Relay2,HIGH);
    bot.sendMessage(CHAT_ID, "Dark Mode Light On");
    delay(1000);
    // SEND_SMS("+919353617078","Dark Mode Light On");
    // delay(2000);
}
if(LDR_Value>500)
{
    digitalWrite(Relay2,LOW);
    delay(1000);
}
}
```

GROUP MEMBERS



NAME	: PREETHA M
USN	: 3PG22CS072
SEM	: 7th SEM
BRANCH	: Computer Science and Engineering
E-MAIL	: mpreetha425@gmail.com



NAME	: S MADHU
USN	: 3PG22CS088
SEM	: 7th SEM
BRANCH	: Computer Science and Engineering
E-MAIL	: saravananmadhu02468@gmail.com



NAME	: SAHANA DIVATAR
USN	: 3PG22CS092
SEM	: 7th SEM
BRANCH	: Computer Science and Engineering
E-MAIL	: sahanadivatar77@gmail.com



NAME	: SANDHYA V
USN	: 3PG22CS095
SEM	: 7th SEM
BRANCH	: Computer Science and Engineering
E-MAIL	: sandhyaddiyaa9999@gmail.com

